

Graph Theory

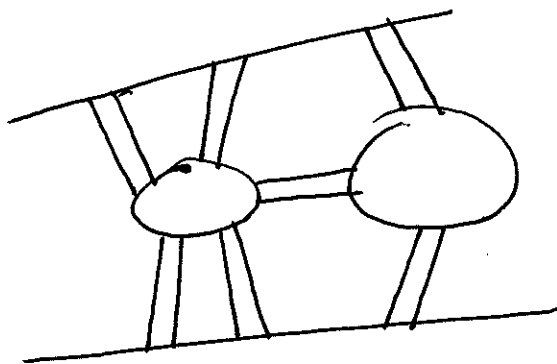
Def. ① Graphs are mathematical structures used to model pairwise relations between objects

② Graph Theory is the study of Graphs.

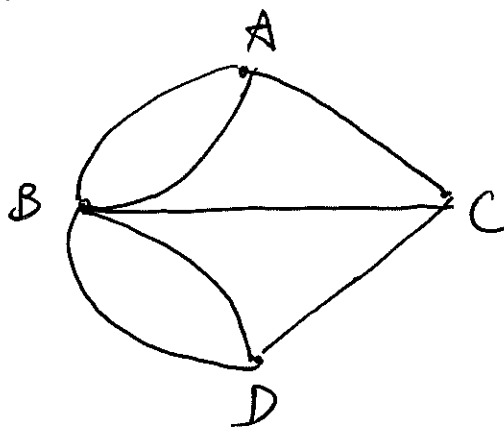
Notation:

A Graph $G = (V, E)$ where V is the set of vertices
& E is the set of edges

History: The first paper in the history of graph theory was published in 1736 by Leonhard Euler
"seven bridges of Königsberg"



find out a walk through the city that would cross each bridge exactly once.

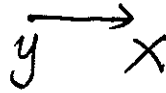


* vertex • vertices ::

* an edge contains two vertices and a line between them.
can be directed or undirected

- an undirected edge is an edge that has no orientation.
so edge (x, y) and edge (y, x) are the same.

- a directed edge is an edge that has orientation
so edge (x, y) and edge (y, x) are different

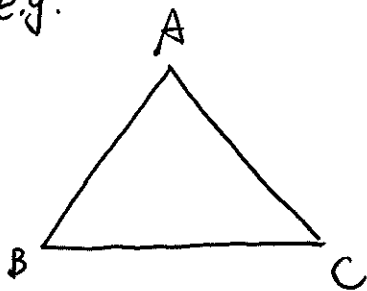


* An undirected graph is a ~~graph~~ graph in which edges have no orientation.

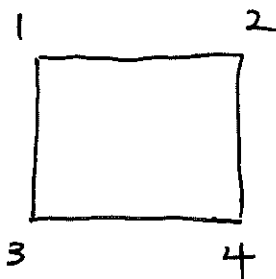
* A directed graph is a graph in which edges have orientations.

* A complete graph is an undirected graph in which every pair of distinct vertices is connected by a unique edge.

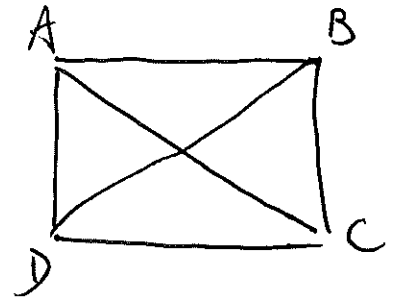
e.g.



✓



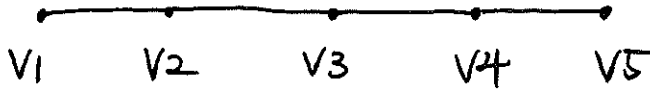
✗



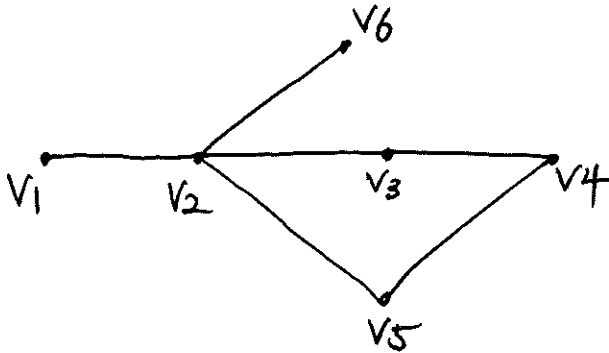
✓

* A path is a finite or infinite sequence of edges which connects a sequence of vertices which are all distinct from one another.

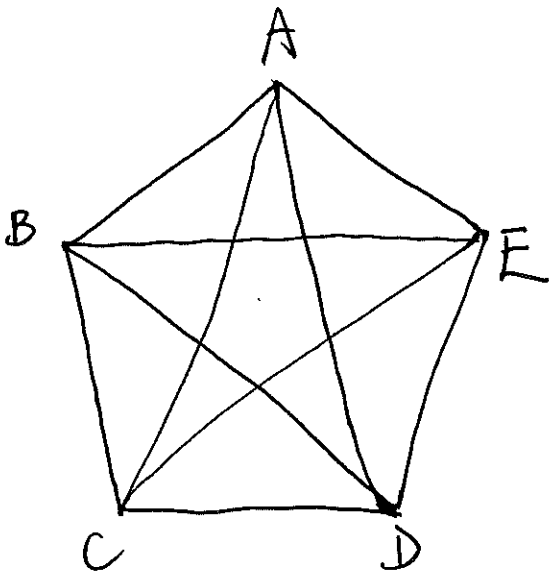
e.g.



$(V_1, V_2, V_3, V_4, V_5)$ is a path



$(V_1, V_2, V_3, V_4, V_5, V_2, V_6)$ is NOT a path



(A, B, C, E) ✓

(A, B, E, C, B, D) ✗

Graph Algorithms

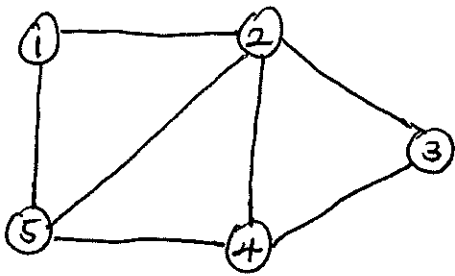
* Representations of graphs

why? --- programming

1. Adjacency-list

2. Adjacency-matrix

e.g. an undirected graph



$G(V, E)$

$V = \{1, 2, 3, 4, 5\}$

$E = \{(1, 2), (2, 3), (3, 4), (4, 5), (5, 1), (2, 4), (2, 5)\}$

Adjacency-list

1 → 2 → 5

2 → 1 → 5 → 4 → 3

3 → 2 → 4

4 → 5 → 2 → 3

5 → 1 → 2 → 4

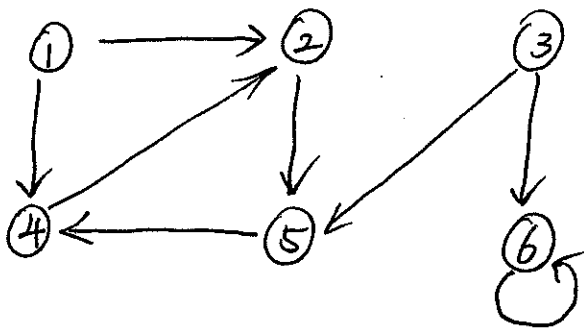
14 connections

7 edges

Adjacency - Matrix

	1	2	3	4	5
1	0	1	0	0	1
2	1	0	1	1	1
3	0	1	0	1	0
4	0	1	1	0	1
5	1	1	0	1	0

e.g. a directed graph



Adjacency - List

1 → 2 → 4

2 → 5

3 → 5 → 6

4 → 2

5 → 4

6 → 6

8 connections

8 edges

Adjacency - Matrix

	1	2	3	4	5	6
1	0	1	0	1	0	0
2	0	0	0	0	1	0
3	0	0	0	0	1	1
4	0	1	0	0	0	0
5	0	0	0	1	0	0
6	0	0	0	0	0	1

* Adjacency-List

Given a graph $G(V, E)$, an adjacency-list representation of G can be created by the following steps

1° for each $V_i \in V$, create a linked list L_i where v_i is the head

2° $\forall V_j \in V$, V_j is in L_i if $(i, j) \in E$

How many linked lists? $|V|$

* Adjacency-Matrix

Given a graph $G(V, E)$, a $|V| \times |V|$ 2-D array $A = (a_{ij})$ can be created such that

$$a_{ij} = \begin{cases} 1 & \text{if } (i, j) \in E \\ 0 & \text{otherwise} \end{cases}$$

Then A is an ~~Adj~~ Adjacency-Matrix of G .

e.g. $|V| = 3$

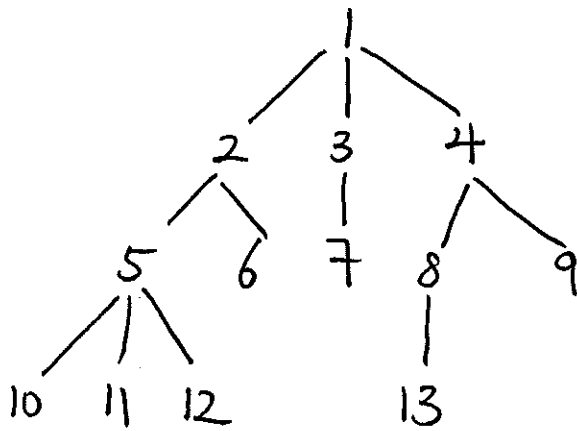
	1	2	3
1	a_{11}	a_{12}	a_{13}
2	a_{21}	a_{22}	a_{23}
3	a_{31}	a_{32}	a_{33}

Graph Traversal / Graph Search

Graph Traversal refers to the process of visiting each vertex in a graph exactly once.

* Breadth-first Search (BFS)

e.g. (intuitive idea) Tree Traversal



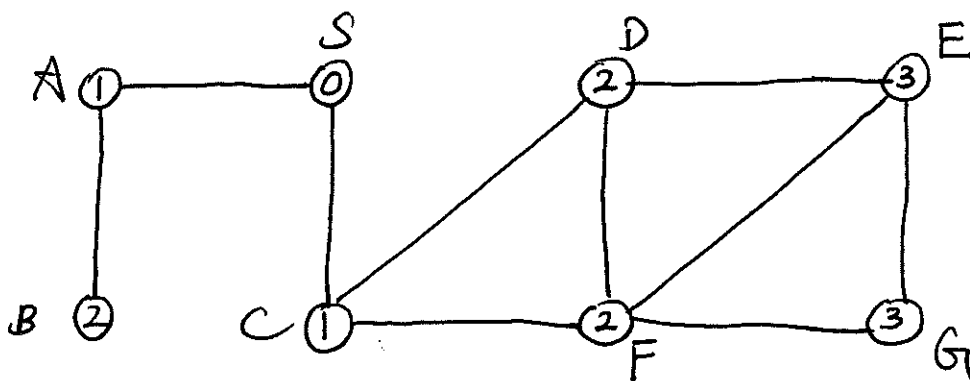
More precise:

The algorithm uses three colors:

- white ... not visited
- gray ... visiting
- black ... visited

& a Queue (queue of gray vertices)

e.g.



initial state: all vertices are white

step 1: start with a random vertex, e.g. S

S.color = gray

Queue.push(S)

Queue

S	
---	--

step 2: visit the front vertex of the Queue, (S in this example)

check the adjacency list L_S

($A, C \in L_S$ and A, C are white)

A.color = gray

Queue.push(A)

C.color = gray

Queue.push(C)

S.color = black

Queue.pop()

$\leftarrow S$
Queue

A	C	
---	---	--

step 3: repeat step 2 until the queue is empty

$\leftarrow A$

C	B	
---	---	--

$\leftarrow C$

B	D	F
---	---	---

$\leftarrow B$

D	F	
---	---	--

$\leftarrow D$

F	E	
---	---	--

$\leftarrow F$

E	G	
---	---	--

$\leftarrow E$

E	G	
--------------	---	--

$\leftarrow G$

--	--	--

BFS ($G(V, E)$, s)

Queue $Q = \text{empty}$;

for each $v \in V$ {

$v.\text{color} = \text{white}$;

$v.\text{depth} = \infty$;

}

$s.\text{color} = \text{gray}$;

$s.\text{depth} = 0$;

$Q.\text{push}(s)$;

while $Q \neq \text{empty}$ {

$x = Q.\text{front}()$;

$Q.\text{pop}()$;

 for each $v \in \text{adjacency list } L_x$ {

 if $v.\text{color} == \text{white}$ {

$v.\text{color} = \text{gray}$

$Q.\text{push}(v)$

$v.\text{depth} = x.\text{depth} + 1$

 }

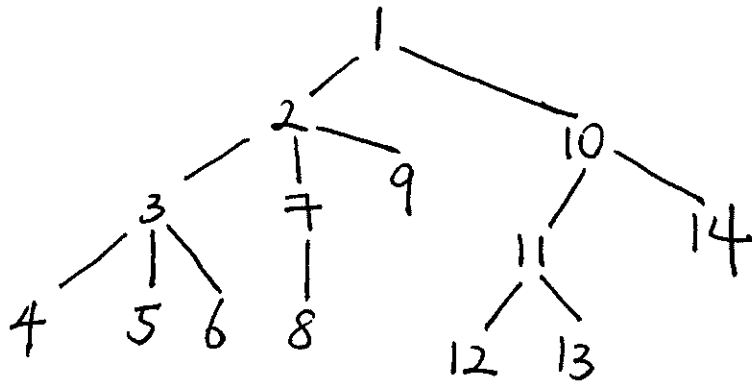
 }

$x.\text{color} = \text{black}$;

}

* Depth-first Search (DFS)

intuitive idea tree traversal



More precise: white -- not visited
gray --- visiting
black --- visited

DFS ($G(V, E)$)

for each $v \in V$ {
 $v.color = white$;
}

time = 0;

for each $v \in V$ {

 if $v.color == white$ {

 DFS-visit($G(V, E), v$)

 }

}

DFS-visit ($G(V, E)$, v)

++ time;

v .start = time

v .color = gray

for each $x \in \text{Adjacency list } L_v$ {

if (x .color == white) {

DFS-visit ($G(V, E)$, x)

}

}

↖ recursive call

so, before we finish v ,

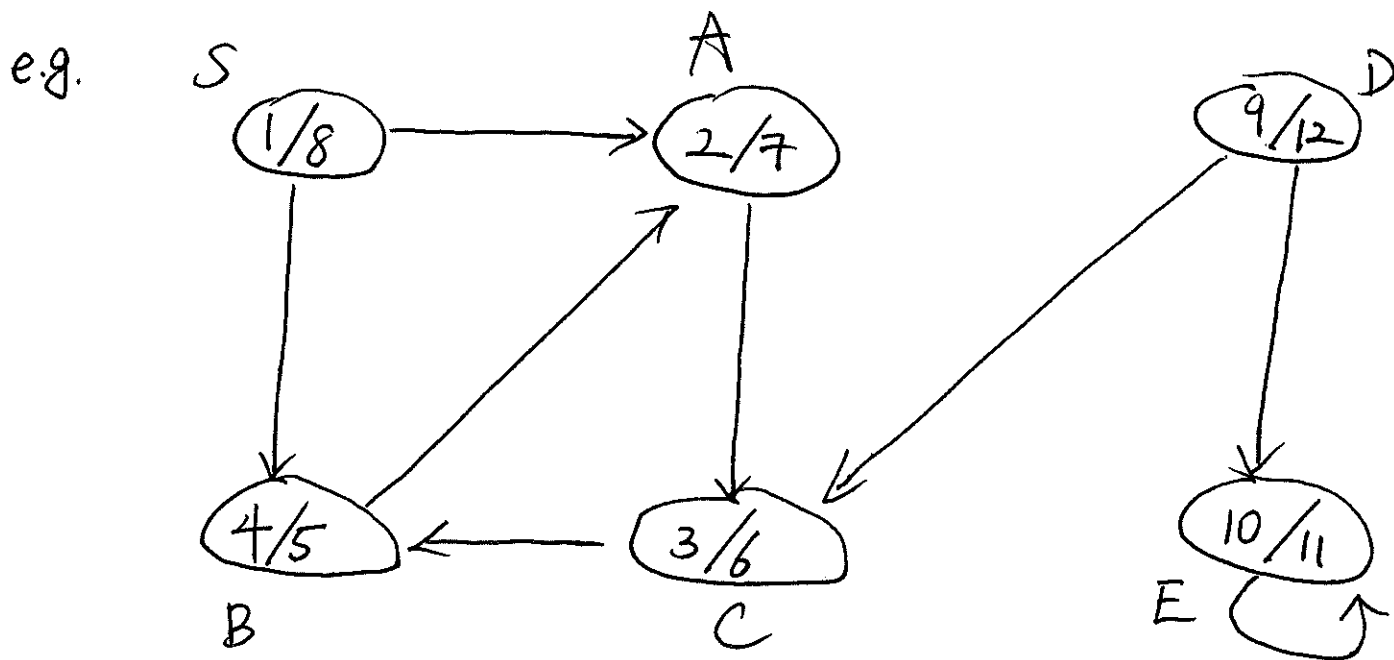
we will finish x first.

(DFS)

v .color = black;

++ time;

v .finish = time;



Time Complexity of BFS / DFS :

1° We traverse each node exactly once. Therefore, the time complexity is at least $O(|V|)$

2° If the graph is implemented using adjacency lists, we can discover all the neighbors of a vertex by traversing the adjacency list of this vertex just once in linear time.

The sum of the sizes of all adjacency lists is $O(|E|)$

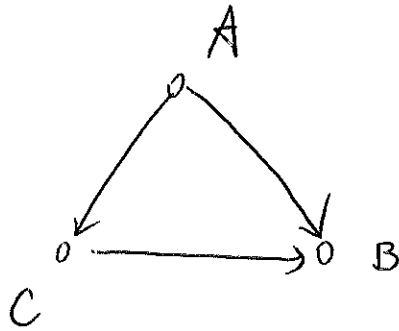
So, the time complexity of DFS/BFS using adjacency list is $O(|V| + |E|)$

Topological Sort

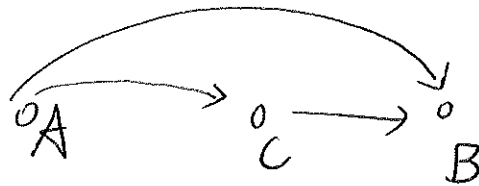
A dag is a directed acyclic graph.

A topological sort of a dag $G=(V,E)$ is a linear ordering of all its vertices such that if G contains an edge (u,v) , then u appears before v in the ordering.

e.g.



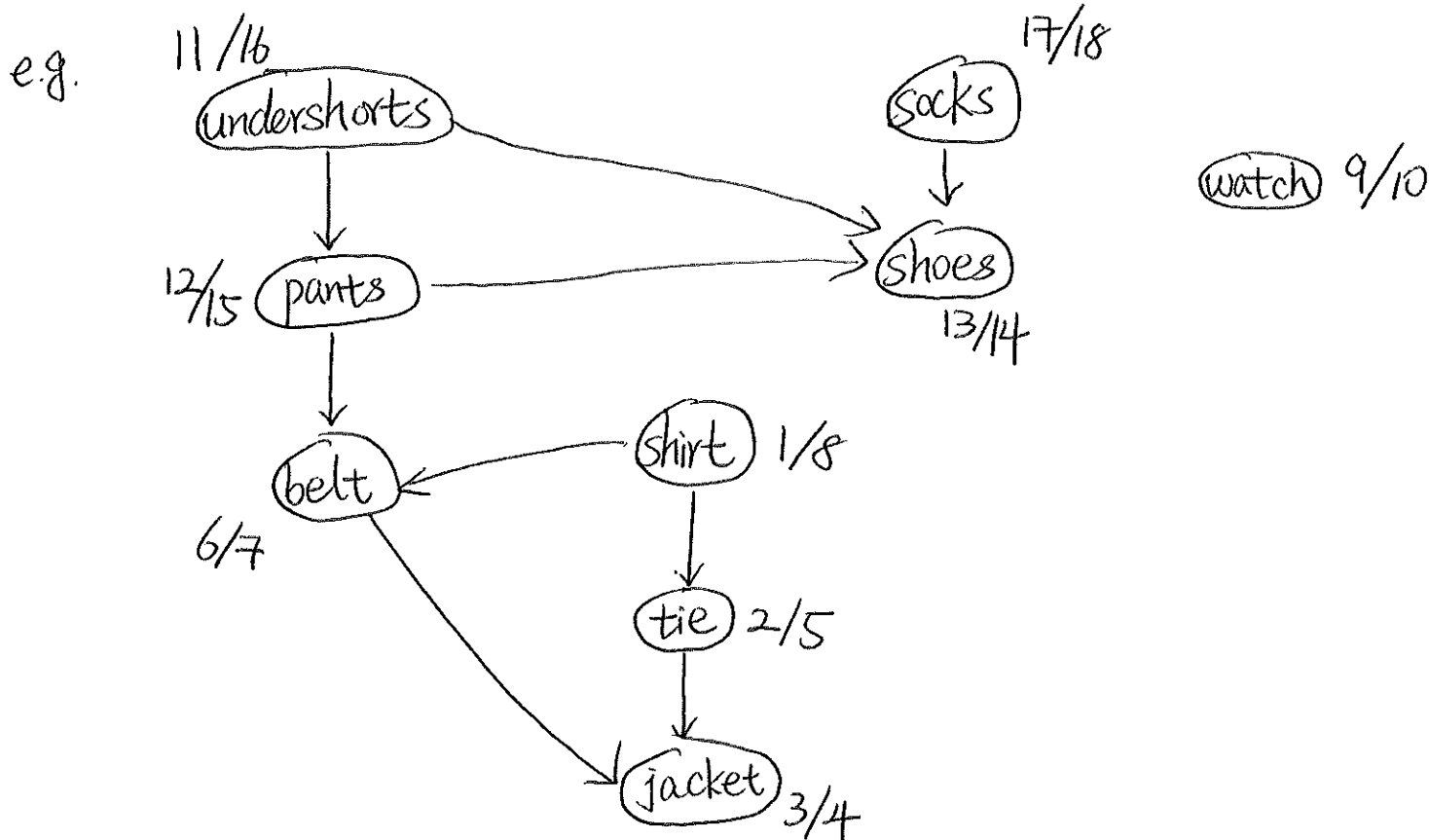
topological sort:



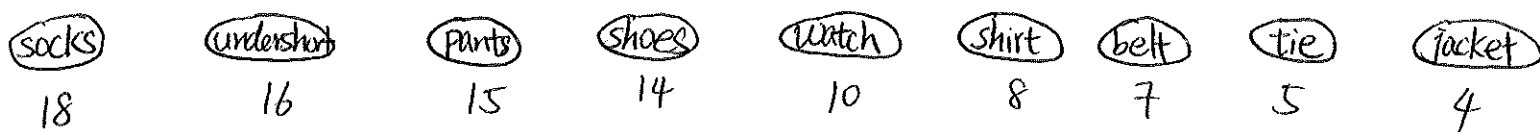
* Topological sorting is only defined on directed graphs without cycles.

Topological-Sort (G):

1. call $\text{DFS}(G)$ to compute finish times $v.f$ for each vertex v
2. as each vertex is finished, insert it onto the front of a linked list (sort by finish time in descending order)
3. return the linked list



sort by finish time in descending order:



check: all directed edges go from left to right

★ Useful lemma

A directed graph G is acyclic if and only if a DFS of G yields no back edges.

(A node points to one of its ancestors)

To find a back edge, we need to use the colors of vertices.

★ If a gray vertex is adjacent to another gray vertex, then the graph has a cycle.

e.g.

